INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

Departamento de Sistemas Digitales · Estrategia de Articulación En Contexto

SolarBI Pascual

Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Trabajo de consulta

Integrantes

Valerie Sofia Cardenas Medina

Juan Sebastian Rodriguez Naranjo

Curso: Inteligencia de Negocios · Grupo 50

Docente: Ramiro Grisales Montoya

Semestre 2026-II

Medellin, Antioquia

Parte A · Consulta

Bloque 1 · Power BI y Grafana

D1. ¿Qué es Grafana? Componentes y Grafana OSS vs. Grafana Cloud

Grafana es una plataforma de código abierto de Grafana Labs para consultar, visualizar y alertar sobre datos, sobre todo series de tiempo, leyéndolos directamente del origen.

Sus componentes son:

- Data sources: Son conectores al origen (en SolarBI sería PostgreSQL)
- Panels: cada visual (time series, stat, gauge) con su consulta, unidades y umbrales
- Dashboards: Son conjuntos de paneles que comparten rango de tiempo y refresco
- Variables: Parámetros como `$dispositivo` que reescriben las consultas y crean listas desplegables
- Alerting, reglas que evalúan una consulta periódicamente y notifican, por ejemplo, potencia en 0 con irradiancia mayor a 300 W/m².

Grafana OSS es gratuito y lo instalamos nosotros; Grafana Cloud es el servicio administrado, con un plan gratuito limitado y planes de pago.

Fuente(s): Grafana Labs (s. f.-a; s. f.-d; s. f.-e).

D2. Tabla comparativa Power BI vs. Grafana

Las dos herramientas no compiten: Power BI modela y explica el negocio; Grafana vigila la operación. En SolarBI ambas leen la misma base PostgreSQL.

Criterio	Power BI	Grafana
Propósito principal	Inteligencia de negocios: informes y cuadros de mando analíticos	Monitoreo y observabilidad casi en tiempo real
Usuario típico	Analista BI, directivos, product owner	Operador de planta, mantenimiento, DevOps
Conexión a los datos	Import, DirectQuery o compuesto; transforma con Power Query	Consulta en vivo al origen en cada refresco; no almacena los datos
Modelo semántico y lenguaje	Modelo tabular en estrella con relaciones; medidas en DAX	Sin modelo semántico: cada panel usa el lenguaje del origen (SQL, PromQL, Flux)
Actualización y tiempo real	Actualización programada: 8/día en Pro, 48/día en Premium	Auto-refresh del dashboard (5 s, 1 min…); ideal para lecturas cada 5 min
Alertas	Alertas sobre tarjetas/KPI en el servicio y Data Activator (Fabric)	Motor de alertas nativo: reglas, umbrales, puntos de contacto
Licenciamiento y costo	Desktop gratis; Pro USD 14 y PPU USD 24 por usuario/mes para compartir	OSS gratis (AGPLv3); Cloud Free; Cloud Pro desde USD 19/mes + uso
Control de acceso	Áreas de trabajo, roles, seguridad a nivel de fila (RLS) en DAX, Microsoft Entra ID	Organizaciones, equipos y permisos por carpeta o dashboard; el filtrado por fila depende del origen
Despliegue	Servicio en la nube de Microsoft o Report Server	Local (instalador o Docker), servidor propio o Cloud
Versionamiento	.pbix binario; .pbip en texto para Git	Dashboard = JSON; provisioning como código
Rol en SolarBI	Desempeño y ahorro (Entregable V)	Operación de inversores cada 5 min

Fuente(s): Microsoft (s. f.-a; s. f.-b; s. f.-e); Grafana Labs (s. f.-a; s. f.-e).

D3. Dashboard operativo, analítico y estratégico

El operativo muestra lo que pasa ahora (minutos u horas) para actuar de inmediato; lo usa el operador. El analítico por otra parte explora tendencias y causas en días o meses, con filtros y comparaciones; lo usa el analista o ingeniero. El estratégico resume pocos KPIs frente a metas (mes o año) para decidir; lo usa el directivo.

Como regla, lo que exige reacción en minutos va a Grafana y lo que exige cálculo y comparación va a Power BI.

KPI mínimo	Tipo	Herramienta	Por qué
Energía (kWh) / potencia	Operativo (potencia 5 min) y analítico (kWh diarios)	Ambas	Grafana para la curva de potencia; Power BI para la energía diaria y mensual
Yield (kWh/kWp) y Performance Ratio	Analítico	Power BI	Requieren agregación diaria, potencia instalada e irradiación: medidas DAX y comparación entre inversores
Ahorro (COP)	Estratégico	Power BI	Combina energía con un parámetro de tarifa; es el lenguaje del directivo
Disponibilidad y alarmas	Operativo	Grafana	Auto-refresh y motor de alertas con umbrales
% de datos válidos	Operativo y de gobernanza	Grafana (y tarjeta en Power BI)	Enfoque G50: el gauge con umbrales avisa de inmediato si cae la calidad; en Power BI da confianza a la cifra

Fuente(s): Kimball y Ross (2013); IEC (2021).

D4. Modos de almacenamiento (Import / DirectQuery) y actualización programada

Import copia los datos al modelo en memoria: consultas muy rápidas y todo DAX disponible, pero los datos sólo cambian al actualizar. DirectQuery deja los datos en PostgreSQL y consulta en cada interacción: más actuales, pero visuales más lentos, carga sobre la base y limitaciones de DAX.

La actualización programada se configura en el servicio (con puerta de enlace para un PostgreSQL local): hasta 8 veces al día con Pro. En SolarBI usaríamos Import con actualización diaria a las 00:30, después del ETL de medianoche: `fact_energia_dia` es diaria, pequeña y su tablero es analítico. Grafana, en cambio, no guarda datos: su auto-refresh vuelve a ejecutar el SQL sobre `silver.lectura_5min` y muestra la última lectura.

Fuente(s): Microsoft (s. f.-a; s. f.-b); Grafana Labs (s. f.-a).

D5. Variables de Grafana vs. segmentadores y filtros de Power BI

Una **variable** de Grafana se define en el dashboard y su valor se inserta como texto dentro de la consulta antes de enviarla: el filtro lo ejecuta la base de datos. Un **segmentador o filtro** de Power BI (en Import) no reescribe el SQL; filtra el modelo ya cargado y el contexto de filtro viaja por las relaciones hasta las medidas DAX.

Proponemos la variable `dispositivo` de tipo Query, multivalor y con opción All, alimentada desde `dwh.dim_dispositivo`. El panel agrega `AND dispositivo_id IN ($dispositivo)`; al elegir Inversor 2, Grafana envía `IN (2)`, y con All, `IN (1,2)`. Su equivalente en Power BI es un segmentador sobre `dim_dispositivo[nombre]`.

```
-- Variable: SELECT dispositivo_id AS __value, nombre AS __text FROM dwh.dim_dispositivo;
SELECT ts AS "time", irradiancia_wm2, p_ac_kw
FROM silver.lectura_5min
WHERE $__timeFilter(ts) AND dispositivo_id IN ($dispositivo)   -- se convierte en IN (2)
ORDER BY 1;
```

Fuente(s): Grafana Labs (s. f.-b; s. f.-c).

Bloque 2 · Gobernanza de datos

G1. Data governance, data management y data quality (DAMA-DMBOK)

Según DAMA-DMBOK, la gobernanza de datos es el ejercicio de autoridad, control y toma de decisiones compartida (planear, vigilar y hacer cumplir) sobre la gestión de los activos de datos; es el centro de la rueda DAMA.

La gestión de datos (data management) es el conjunto de prácticas que planean y ejecutan el ciclo de vida de los datos: integrar, modelar, almacenar, proteger y entregar.

La calidad de datos es una de esas áreas: mide y mejora que los datos sean aptos para su uso. En SolarBI, gobernar es decidir que la meta es 95 % de datos válidos y quién la aprueba; gestionar es `run_etl.py`; la calidad son las reglas R1–R3 y el 94 % medido.

Fuente: DAMA International (2017).

G2. Roles data owner, data steward y data custodian

El data owner es el responsable de negocio que rinde cuentas por un dominio de datos: aprueba definiciones, accesos y niveles de calidad.

El data steward cuida a diario el significado y la calidad: mantiene el diccionario y las reglas, y atiende incidentes.

El data custodian es técnico: opera la infraestructura (bases, permisos, copias, ETL) siguiendo las políticas que otros definen.

Rol en el proyecto	Rol de gobernanza	Responsabilidad en SolarBI
Product owner	Data owner	Dueño de los KPIs; aprueba el contrato de datos y la meta de 95 % de datos válidos
BI analyst	Data steward (indicadores)	Mantiene el catálogo de KPIs y que cada fórmula exista una sola vez
Data modeler	Data steward (modelo)	Esquema estrella, claves, diccionario de datos y unidades
Data engineer	Data custodian	PostgreSQL, ETL idempotente, reglas R1–R3, credenciales en .env

Fuente: DAMA International (2017).

G3. El contrato de datos como instrumento de gobernanza

El contrato de datos es un acuerdo formal y versionado entre quien produce los datos (inversor y gateway) y quien los consume (ETL y tableros). Debe fijar esquema, unidades, frecuencia, rangos válidos, responsable, niveles de calidad y política de cambios. Si la fuente cambia un campo sin avisar, por ejemplo *p_ac_kw* por *p_ac_w*, en el mejor caso el ETL falla; en el peor, carga vatios como kilovatios, la regla R3 rechaza casi todo, el % de datos válidos se desploma o la energía y el ahorro se multiplican por mil, y el directivo decide con cifras falsas. Por eso el ETL debe validar el esquema antes de cargar y fallar de forma visible.

Elemento	Contrato SolarBI (resumen)
Esquema	ts (timestamp, hora local UTC−5), dispositivo_id (entero), p_ac_kw, irradiancia_wm2, temp_modulo_c (decimales). Clave: ts + dispositivo_id
Unidades y rangos	kW: 0 a 5,5 · W/m²: 0 a 1.500 · °C: −10 a 90
Frecuencia	Cada 5 minutos: 288 lecturas por día y dispositivo; ETL diario a las 00:00
Responsable	Owner: product owner · Contacto técnico: data engineer
Niveles de calidad	≥ 95 % de lecturas válidas por día; 0 duplicados en Silver
Cambios	Versión semántica (v1.0); todo cambio de campo o unidad se avisa antes y sube la versión

Fuentes: DAMA International (2017);

G4. Única fuente de verdad y catálogo de KPIs

La única fuente de verdad es un repositorio gobernado del que leen todos los tableros, para que cada cifra tenga un único valor oficial; en SolarBI es el esquema *dwh* de PostgreSQL.

El catálogo de KPIs documenta la definición oficial, fórmula, dueño y frecuencia de cada indicador. El riesgo de calcular un KPI en DAX y otra vez en SQL lo vivimos en la práctica: el % de datos válidos dio 94,00 % sobre filas leídas (incluye duplicados) y 96,18 % sobre las 288 lecturas esperadas. Ambas cifras son “correctas”, pero operador y directivo verían números distintos y perderían confianza.

Solución: calcular el KPI una vez en Gold y que DAX y Grafana solo lo agreguen.

Campo	KPI 1 · Yield (rendimiento específico)	KPI 2 · % de datos válidos
Fórmula	Σ energia_kwh / Σ potencia_kwp instalada	Σ lecturas_validas / Σ lecturas_esperadas × 100
Unidad	kWh/kWp (del periodo; también kWh/kWp/día)	%
Fuente	dwh.fact_energia_dia + dwh.dim_dispositivo	dwh.fact_energia_dia (lo calcula run_etl.py)
Responsable	BI analyst (cálculo) · product owner (aprueba)	Data engineer (mide) · data steward (vigila)
Frecuencia	Diaria, tras el ETL de las 00:00	Diaria; en Grafana, en cada refresco
Meta / referencia	Valor de la práctica: 5,30 kWh/kWp/día	≥ 95 % (verde), 90–95 % (amarillo), < 90 % (rojo)

Fuentes: DAMA International (2017); IEC (2021); Kimball y Ross (2013).

G5. Ley 1581 de 2012 y manejo responsable de la información

La Ley 1581 define dato personal como cualquier información vinculada o que pueda asociarse a personas naturales determinadas o determinables. La telemetría de SolarBI (potencia, irradiancia, temperatura) describe equipos, no personas: no es un dato personal. Si el tablero mostrara qué operador atendió cada alarma, aparecería un dato personal que, además, permitiría evaluar el desempeño del trabajador. Se necesitaría autorización previa e informada, finalidad declarada, acceso restringido y respeto a los derechos del titular a conocer, actualizar y rectificar.

Nuestra postura: minimizar; mostrar solo un código de turno, dejar los nombres en el sistema de mantenimiento con acceso restringido, no subir datos personales ni credenciales a GitHub y usar los datos para mejorar la planta, no para vigilar personas.

Fuente: Congreso de la República de Colombia (2012), arts. 3, 4, 8 y 9.

Bloque 3 · Automatización de procesos ETL

E1. ETL vs. ELT en las capas Bronze, Silver y Gold. ¿Power Query es ETL?

En ETL los datos se extraen, se transforman fuera del destino y se cargan ya limpios. En ELT se cargan crudos en el destino (lago o almacén) y se transforman allí con su propio motor, normalmente SQL. Nuestro flujo es híbrido: pandas limpia antes de cargar Silver (ETL) y la agregación a Gold podría hacerse en PostgreSQL (ELT).

¿Power Query es ETL? Sí, de autoservicio: extrae de muchos orígenes, transforma con M y carga al modelo. Pero sus pasos viven dentro de un .pbix, Grafana no los ve y no tiene orquestación ni bitácora. Por gobernanza, las reglas de calidad van en el pipeline central y Power Query solo hace ajustes de presentación.

Capa	Paso	Objeto en SolarBI
Bronze	E: copia cruda e inmutable de la fuente	data/bronze/telemetria.csv
Silver	T: R1 duplicados, R2 faltantes, R3 rango; energía = p × 5/60	silver.lectura_5min (1.662 filas)
Gold	T + L: agregación diaria y modelo estrella	dwh.fact_energia_dia, dim_fecha, dim_dispositivo

Fuentes: Databricks (s. f.); Microsoft (s. f.-d); Kimball y Ross (2013).

E2. Carga completa, carga incremental e idempotencia (UPSERT)

La carga completa borra y recarga todo en cada ejecución: es simple, pero cada vez más costosa y deja la tabla vacía mientras corre.

La carga incremental solo trae lo nuevo o modificado desde la última marca de agua (por ejemplo, el *max(ts)* cargado): es eficiente, pero exige controlar duplicados y datos que llegan tarde.

Una carga es idempotente cuando ejecutarla una o diez veces deja el mismo resultado. En *fact_energia_dia* se logra con la clave primaria (*fecha_key, dispositivo_key*) y un UPSERT: si la fila existe se actualiza; si no, se inserta. Lo comprobamos: dos ejecuciones dejaron 6 filas en el hecho y 1.662 en Silver.

```
INSERT INTO dwh.fact_energia_dia (fecha_key, dispositivo_key, energia_kwh, ..., pct_datos_validos)
VALUES %s
ON CONFLICT (fecha_key, dispositivo_key) DO UPDATE SET
    energia_kwh = EXCLUDED.energia_kwh, pct_datos_validos = EXCLUDED.pct_datos_validos, ...;
```

Fuentes: PostgreSQL Global Development Group (s. f.); Kimball y Ross (2013).

E3. Tabla comparativa: cron / Programador de tareas, Apache Airflow y actualización programada de Power BI

Para SolarBI hoy basta con cron (o el Programador de tareas) a las 00:00 y la actualización programada de Power BI a las 00:30. Airflow convendrá cuando haya varios sitios, dependencias y necesidad de reintentos.

	cron / Programador de tareas	Apache Airflow	Actualización programada de Power BI
Qué es	Programador del sistema operativo que lanza comandos a una hora	Plataforma open source para definir, programar y monitorear flujos como DAG en Python	Función del servicio Power BI que recarga un modelo semántico publicado
Qué automatiza	Un comando: python etl/run_etl.py	Tareas con dependencias, reintentos, backfill, alertas y registros	Solo la recarga del modelo (consultas de Power Query)

	cron / Programador de tareas	Apache Airflow	Actualización programada de Power BI
Monitoreo	Archivo de log propio	Interfaz web con estado de cada tarea	Historial de actualización y correo si falla
Cuándo conviene	Un script, un servidor, proyecto pequeño	Muchos flujos y orígenes; equipo de datos	Cuando el tablero debe reflejar la base tras el ETL; no reemplaza el ETL

Fuentes: The Apache Software Foundation (s. f.); Microsoft (s. f.-b).

E4. Actualización incremental en Power BI (RangeStart y RangeEnd)

Se crean dos parámetros reservados de tipo fecha/hora, RangeStart y RangeEnd, y en Power Query se filtra la tabla con `ts >=` *RangeStart* y `ts <` *RangeEnd* (una sola desigualdad inclusiva, para no duplicar filas en los bordes). Luego se define la política: por ejemplo, conservar 2 años y actualizar solo los últimos 7 días. En el servicio, Power BI divide la tabla en particiones por fecha y sustituye los parámetros en cada una: solo vuelve a consultar las recientes y conserva las históricas. Requiere que el filtro se traduzca a un `WHERE` en PostgreSQL (query folding). Vale la pena con tablas grandes, como *lectura_5min* a lo largo de años; para *fact_energia_dia*, recargar todo es más simple.

```
Filtrado = Table.SelectRows(Lecturas, each [ts] >= RangeStart and [ts] < RangeEnd)
```

Fuente: Microsoft (s. f.-c).

Bloque 4 · Internet de las Cosas (IoT)

I1. Arquitectura IoT por capas

- **Dispositivo:** el inversor y el piranómetro miden y exponen registros.
- **Gateway (edge):** un datalogger o una Raspberry Pi los lee cada 5 minutos, arma un JSON y lo guarda en búfer si se cae la red.
- **Red o broker:** publica por MQTT al broker, que reparte a los suscriptores.
- **Almacenamiento:** un suscriptor de ingesta escribe el crudo (Bronze); el ETL limpia (Silver) y agrega por día (Gold).
- **Aplicación:** Grafana lee Silver para la operación y Power BI lee Gold para el desempeño y el ahorro. Una lectura es Bronze al llegar, Silver si pasa R1–R3 y parte de Gold cuando se suma al total diario.

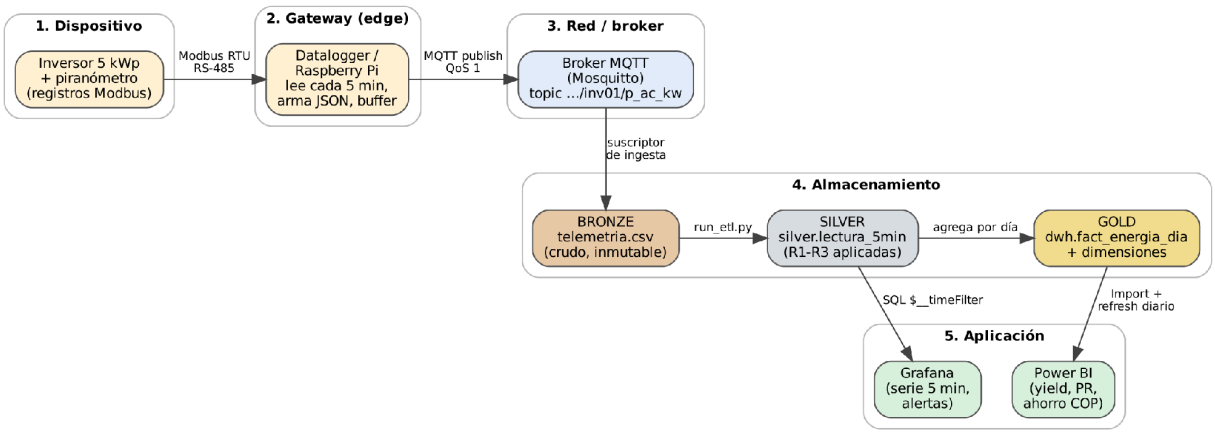


Figura 1. Recorrido de una lectura del inversor al tablero.

Fuentes: OASIS (2019); Databricks (s. f.).

I2. MQTT: publish/subscribe, broker, topic y QoS

MQTT es un protocolo de mensajería liviano (estándar OASIS, sobre TCP) pensado para dispositivos limitados y redes inestables.

En el modelo publish/subscribe, el gateway publica en un topic sin conocer a los consumidores, y el broker (por ejemplo, Mosquitto) entrega cada mensaje a los suscritos.

Los topics son jerárquicos, con «/», y admiten comodines + (un nivel) y # (varios niveles).

QoS 0: como máximo una vez, sin confirmación.

QoS 1: al menos una vez, con confirmación, pero puede duplicar; por eso existen la regla R1 y el UPSERT.

QoS 2: exactamente una vez, con más intercambio de mensajes.

Para la telemetría usaríamos QoS 1; para el estado del equipo, mensaje retenido.

```
pascualbravo/solar/<sitio>/<dispositivo>/<variable>
pascualbravo/solar/robledo/inv01/p_ac_kw            {"ts":"2026-10-05 12:00:00","valor":4.21}
pascualbravo/solar/robledo/inv01/temp_modulo_c
pascualbravo/solar/robledo/piranometro01/irradiancia_wm2
pascualbravo/solar/robledo/medidor01/energia_kwh
pascualbravo/solar/robledo/inv01/estado            (retain = true)
pascualbravo/solar/robledo/inv01/alarma            (QoS 1)
Suscripciones: pascualbravo/solar/robledo/+/p_ac_kw   |   pascualbravo/solar/#
```

Fuente: OASIS (2019).

I3. Tabla comparativa Modbus, MQTT y OPC UA

	Modbus (RTU / TCP)	MQTT	OPC UA
Modelo de comunicación	Maestro-esclavo (cliente-servidor) por sondeo: pregunta y respuesta	Publish/subscribe con broker	Cliente-servidor y pub/sub, con modelo de información orientado a objetos
Uso típico	Leer registros de equipos de campo: inversores, medidores, PLC	Telemetría de muchos dispositivos hacia la nube	Integración industrial interoperable (SCADA, MES)

	Modbus (RTU / TCP)	MQTT	OPC UA
Formato de los datos	Registros de 16 bits sin semántica; hay que conocer el mapa (p. ej. SunSpec)	Payload libre (JSON, binario); el topic da el contexto	Nodos tipados con metadatos: nombre, unidad, rango
Seguridad	Sin seguridad nativa (texto plano); variante segura con TLS poco adoptada	TLS, usuario/clave o certificados; permisos por topic	Integrada: certificados X.509, firma y cifrado, roles
Transporte	RS-485 serie (RTU) o TCP puerto 502	TCP 1883 / 8883 con TLS	TCP 4840 o HTTPS

En los inversores y medidores de una planta solar esperaríamos sobre todo Modbus RTU (RS-485) y Modbus TCP, muchas veces con el mapa de registros SunSpec. El datalogger traduce esas lecturas a MQTT (o a la API del fabricante) para subirlas. OPC UA aparece en plantas grandes con SCADA.

Fuentes: Modbus Organization (2012); OASIS (2019); OPC Foundation (s. f.).

I4. Procesamiento por lotes (batch) vs. flujo continuo (streaming)

El batch procesa bloques acumulados a intervalos fijos (por ejemplo, cada noche): es simple y eficiente, pero con latencia alta.

El streaming procesa cada evento al llegar, en segundos: baja latencia, pero más complejidad (broker, ventanas de tiempo, estado). Al agregar 288 lecturas de 5 minutos en una fila diaria (en la práctica, 1.662 lecturas quedaron en 6 filas) se gana volumen menor, consultas rápidas y KPIs comparables (yield, PR, ahorro). Se pierde el perfil del día: picos, caídas por nubes, la hora exacta de una falla o el recorte de potencia. Ese detalle no se recupera desde el agregado, por eso Silver se conserva. Los 5 minutos los atiende mejor Grafana; la granularidad diaria, Power BI.

Fuentes: Kimball y Ross (2013); Grafana Labs (s. f.-a); Microsoft (s. f.-a).

Bloque 5 · Control de versiones

V1. Git y GitHub, .pbix vs. .pbip y versionamiento de dashboards de Grafana

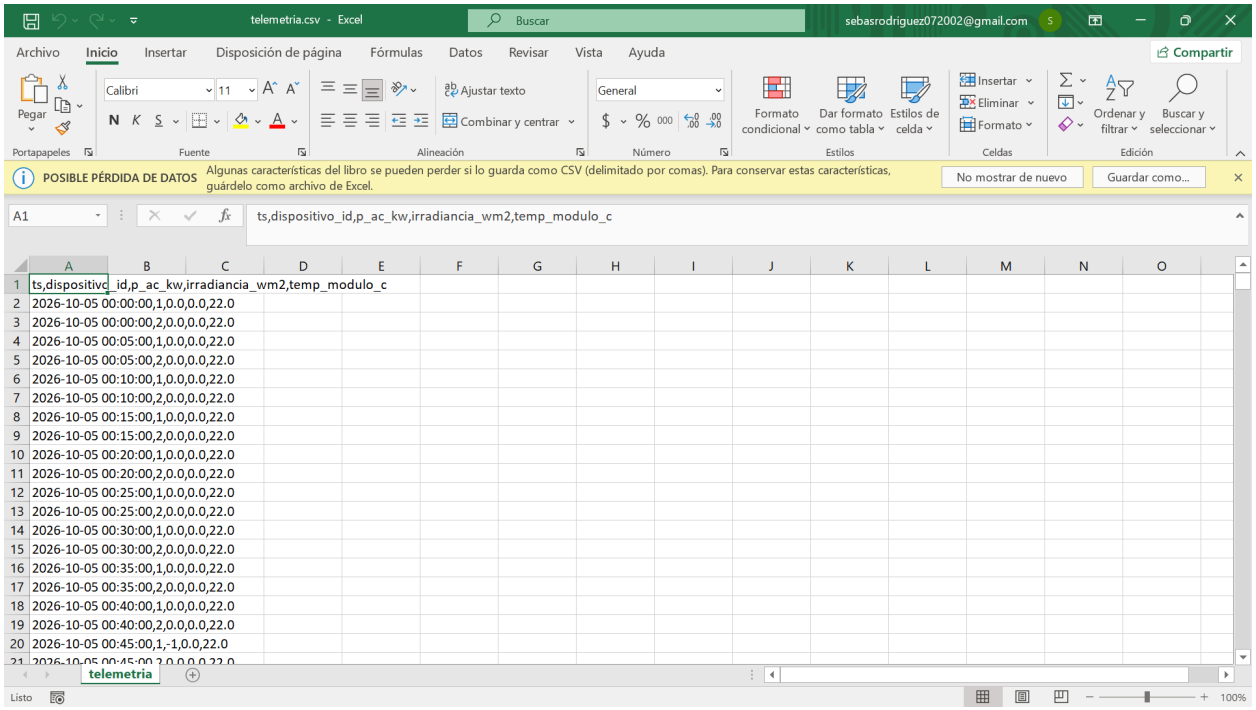
- **Git:** Es el sistema de control de versiones distribuido que guarda localmente el historial.
- **GitHub:** Es la plataforma en la nube que aloja repositorios Git y añade colaboración: permisos, pull requests, issues.
- **Repository:** Carpeta del proyecto con su historial.
- **Commit:** instantánea de cambios con autor, fecha y mensaje.
- **Branch:** línea de trabajo paralela, como *feature/grafana*, que no afecta *main*.
- **Pull request:** solicitud para integrar una rama a *main* con revisión del compañero.

Un .pbix es binario y comprimido: Git no muestra qué cambió ni fusiona el trabajo de dos personas. El .pbip guarda informe y modelo como texto (JSON y TMDL): se ve qué medida DAX cambió. Un dashboard de Grafana se exporta como JSON y se versiona.

Fuentes: Chacon y Straub (2014); Microsoft (s. f.-f); Grafana Labs (s. f.-a).

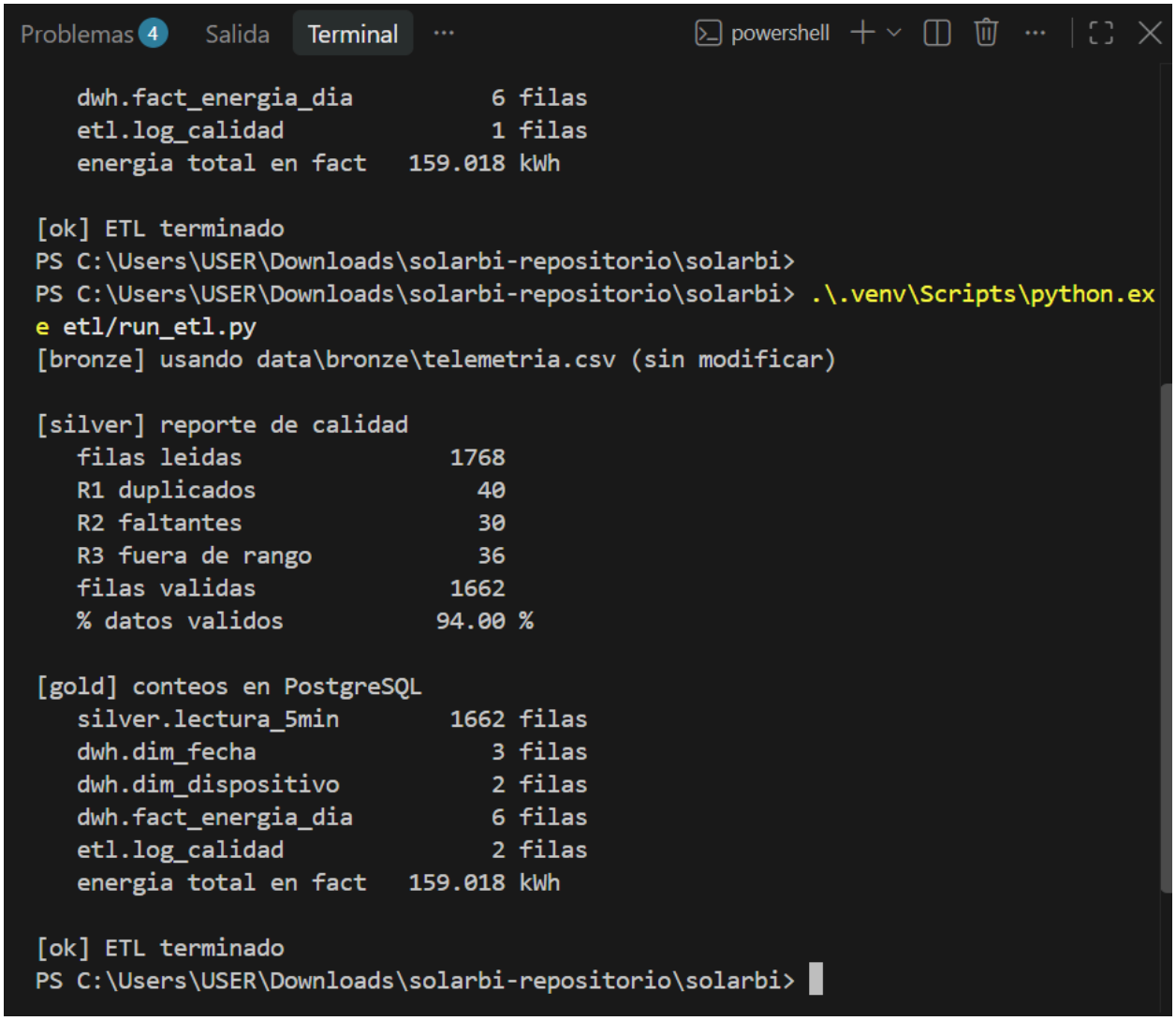
Parte B · Práctica guiada: los mismos datos en dos herramientas

Paso 1 · Bronze: datos crudos



La capa Bronze almacena los datos originales de telemetría de los inversores solares, incluyendo las mediciones registradas cada cinco minutos. Estos datos constituyen la fuente inicial del proceso ETL y se conservan para permitir su posterior limpieza y transformación.

Paso 2 · Silver: reglas de calidad



El proceso ETL leyó 1.768 registros y aplicó reglas para identificar datos duplicados, faltantes y valores fuera de los rangos físicos establecidos. Se obtuvieron 1.662 registros válidos, equivalentes al 94 % del total leído. Este resultado permite evaluar la calidad de los datos antes de utilizarlos en los análisis.

Métrica	Valor
Filas leídas	1.768
Rechazadas · rango físico	36
Rechazadas · duplicados	40
Rechazadas · datos faltantes	30
Filas válidas	1.662
% de datos válidos	94,00 %

Paso 3 · Gold: carga en PostgreSQL

```
Problemas 4  Salida  Terminal  ...  powershell
PS C:\Users\USER\Downloads\solarbi-repositorio> cd .\solarbi
PS C:\Users\USER\Downloads\solarbi-repositorio\solarbi> .\.venv\Scripts\python.ex
e etl/run_etl.py
[bronze] usando data\bronze\telemetria.csv (sin modificar)

[silver] reporte de calidad
   filas leidas              1768
   R1 duplicados               40
   R2 faltantes                30
   R3 fuera de rango           36
   filas validas             1662
   % datos validos          94.00 %

[gold] conteos en PostgreSQL
   silver.lectura_5min       1662 filas
   dwh.dim_fecha                3 filas
   dwh.dim_dispositivo          2 filas
   dwh.fact_energia_dia         6 filas
   etl.log_calidad              1 filas
   energia total en fact   159.018 kWh

[ok] ETL terminado
PS C:\Users\USER\Downloads\solarbi-repositorio\solarbi>
```

```
Problemas 4  Salida  Terminal  ...  powershell
   dwh.fact_energia_dia         6 filas
   etl.log_calidad              1 filas
   energia total en fact   159.018 kWh

[ok] ETL terminado
PS C:\Users\USER\Downloads\solarbi-repositorio\solarbi>
PS C:\Users\USER\Downloads\solarbi-repositorio\solarbi> .\.venv\Scripts\python.ex
e etl/run_etl.py
[bronze] usando data\bronze\telemetria.csv (sin modificar)

[silver] reporte de calidad
   filas leidas              1768
   R1 duplicados               40
   R2 faltantes                30
   R3 fuera de rango           36
   filas validas             1662
   % datos validos          94.00 %

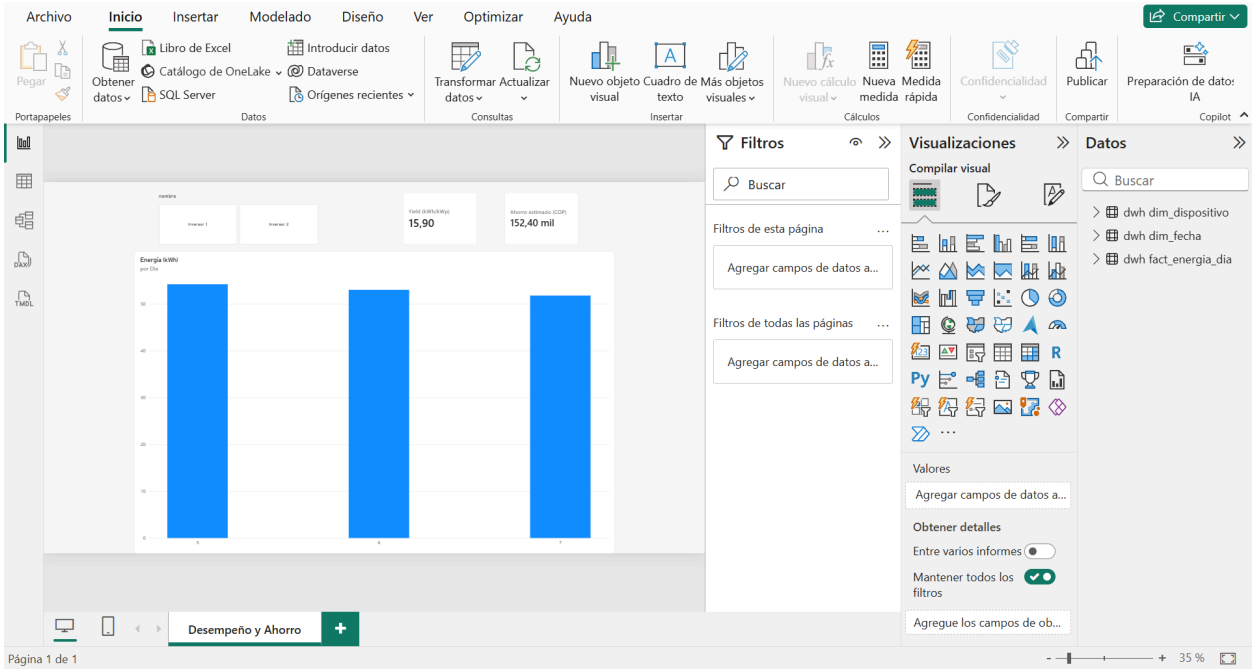
[gold] conteos en PostgreSQL
   silver.lectura_5min       1662 filas
   dwh.dim_fecha                3 filas
   dwh.dim_dispositivo          2 filas
   dwh.fact_energia_dia         6 filas
   etl.log_calidad              2 filas
   energia total en fact   159.018 kWh

[ok] ETL terminado
PS C:\Users\USER\Downloads\solarbi-repositorio\solarbi>
```

Se ejecutó el proceso ETL dos veces para comprobar la carga de los datos en PostgreSQL. En ambas ejecuciones se conservaron los mismos conteos de registros y los mismos resultados de energía, sin duplicar la información de las tablas principales. Esto evidencia el comportamiento idempotente del proceso. La tabla de registro de calidad aumentó de una a dos filas porque almacena cada ejecución.

Expresión cron / Programador de tareas (medianoche): 30 0 * * * — ejecución diaria a las 00:30.

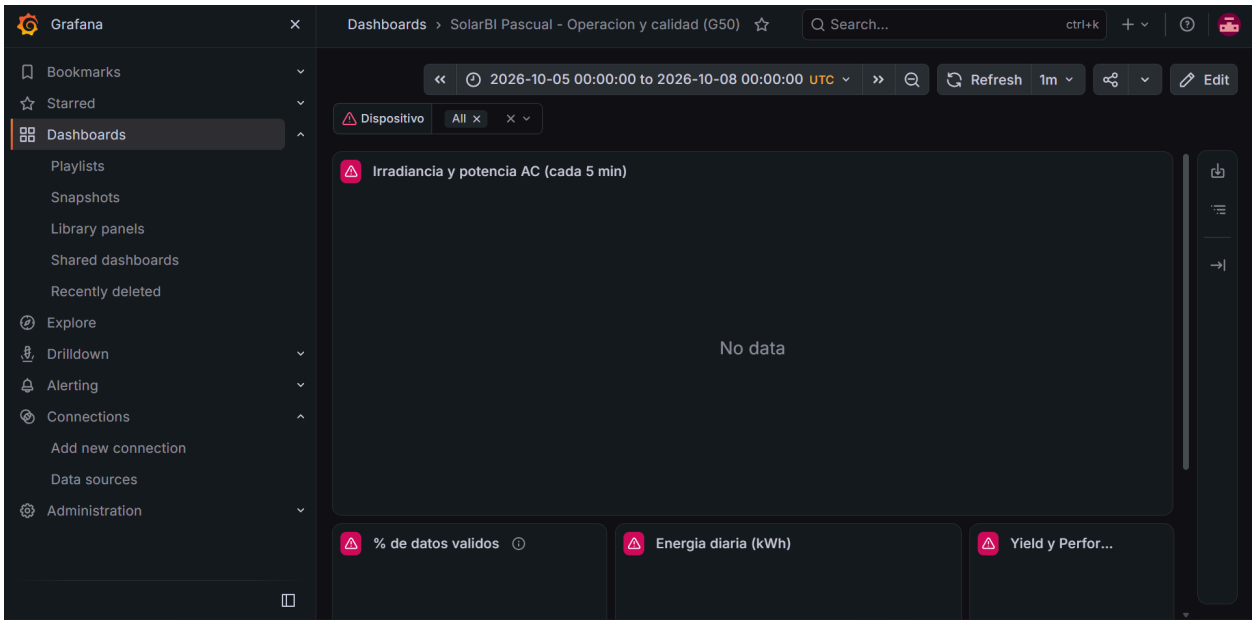
Paso 4 · Power BI



El informe de Power BI presenta indicadores de desempeño de la planta solar mediante medidas y visualizaciones construidas a partir de los datos procesados. La tarjeta de rendimiento, el gráfico de energía diaria y el indicador de ahorro permiten interpretar los resultados de manera resumida y facilitar la toma de decisiones.

Fuente de la tarifa usada: [entidad, fecha, valor en COP/kWh]

Paso 5 · Grafana

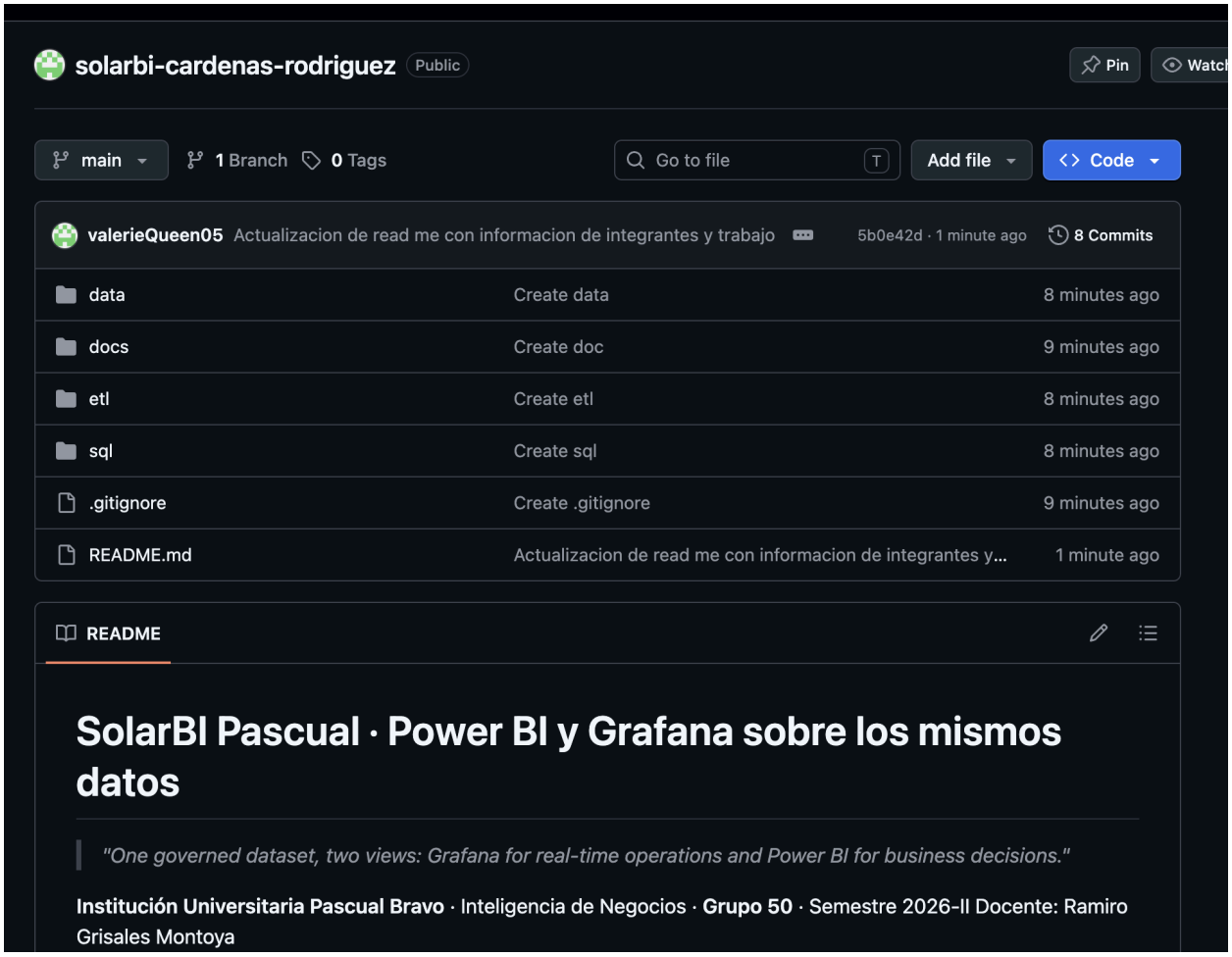


Paso 6 · Reflexión

Con los mismos datos, cada herramienta resolvió mejor una cosa distinta. Grafana fue más útil para ver el detalle cada 5 minutos: la curva de potencia frente a la irradiancia, el indicador de % de datos válidos con colores y el filtro por dispositivo salen directo con SQL y se actualizan solos. Power BI fue mejor para los indicadores de negocio, como el yield, el Performance Ratio y el ahorro en pesos con un parámetro de tarifa, porque el modelo estrella y las medidas DAX permiten comparar días e inversores sin reescribir consultas. Lo que más nos costó en Grafana fueron las macros como $__timeFilter y la zona horaria; en Power BI, que el yield no se sumara mal entre días y dispositivos. También vimos que un mismo KPI puede dar 94 % o 96 % según cómo se calcule, y por eso conviene definirlo una sola vez. Para el operador de la planta usaríamos Grafana, y para un directivo, Power BI.

Parte C · Repositorio en GitHub

Enlace al repositorio: https://github.com/valerieQueen05/solarbi-cardenas-rodriguez



Referencias

Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Congreso de la República de Colombia. (2012, 17 de octubre). Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. Diario Oficial 48.587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

DAMA International. (2017). DAMA-DMBOK: Data management body of knowledge (2.ª ed.). Technics Publications.

Databricks. (s. f.). Medallion architecture. Recuperado el 8 de octubre de 2026 de https://www.databricks.com/glossary/medallion-architecture

Empresas Públicas de Medellín [EPM]. (2026). Publicación de tarifas de energía eléctrica, septiembre de 2026, mercado regulado Antioquia. https://www.epm.com.co/clientesyusuarios/energia/tarifas-energia/

Grafana Labs. (s. f.-a). Grafana documentation: Dashboards and panels. Recuperado el 8 de octubre de 2026 de https://grafana.com/docs/grafana/latest/dashboards/

Grafana Labs. (s. f.-b). Variables. Recuperado el 8 de octubre de 2026 de https://grafana.com/docs/grafana/latest/dashboards/variables/

Grafana Labs. (s. f.-c). PostgreSQL data source. Recuperado el 8 de octubre de 2026 de https://grafana.com/docs/grafana/latest/datasources/postgres/

Grafana Labs. (s. f.-d). Grafana Alerting. Recuperado el 8 de octubre de 2026 de https://grafana.com/docs/grafana/latest/alerting/

Grafana Labs. (s. f.-e). Pricing. Recuperado el 8 de octubre de 2026 de https://grafana.com/pricing/

International Electrotechnical Commission [IEC]. (2021). IEC 61724-1:2021 Photovoltaic system performance – Part 1: Monitoring. IEC.

Kimball, R. y Ross, M. (2013). The data warehouse toolkit: The definitive guide to dimensional modeling (3.ª ed.). Wiley.

Microsoft. (s. f.-a). Semantic model modes in the Power BI service. Microsoft Learn. Recuperado el 8 de octubre de 2026 de https://learn.microsoft.com/en-us/power-bi/connect-data/service-dataset-modes-understand

Microsoft. (s. f.-b). Configure scheduled refresh. Microsoft Learn. Recuperado el 8 de octubre de 2026 de https://learn.microsoft.com/en-us/power-bi/connect-data/refresh-scheduled-refresh

Microsoft. (s. f.-c). Incremental refresh and real-time data for semantic models. Microsoft Learn. Recuperado el 8 de octubre de 2026 de https://learn.microsoft.com/en-us/power-bi/connect-data/incremental-refresh-overview

Microsoft. (s. f.-d). What is Power Query? Microsoft Learn. Recuperado el 8 de octubre de 2026 de https://learn.microsoft.com/en-us/power-query/power-query-what-is-power-query

Microsoft. (s. f.-e). Power BI pricing. Recuperado el 8 de octubre de 2026 de https://www.microsoft.com/en-us/power-platform/products/power-bi/pricing

Microsoft. (s. f.-f). Power BI Desktop projects (PBIP). Microsoft Learn. Recuperado el 8 de octubre de 2026 de https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-overview

Modbus Organization. (2012). MODBUS application protocol specification V1.1b3. https://modbus.org/specs.php

OASIS. (2019). MQTT version 5.0 (OASIS Standard). https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

OPC Foundation. (s. f.). Unified Architecture. Recuperado el 8 de octubre de 2026 de https://opcfoundation.org/about/opc-technologies/opc-ua/

PostgreSQL Global Development Group. (s. f.). INSERT (ON CONFLICT clause). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-insert.html

The Apache Software Foundation. (s. f.). Apache Airflow documentation. Recuperado el 8 de octubre de 2026 de https://airflow.apache.org/docs/apache-airflow/stable/